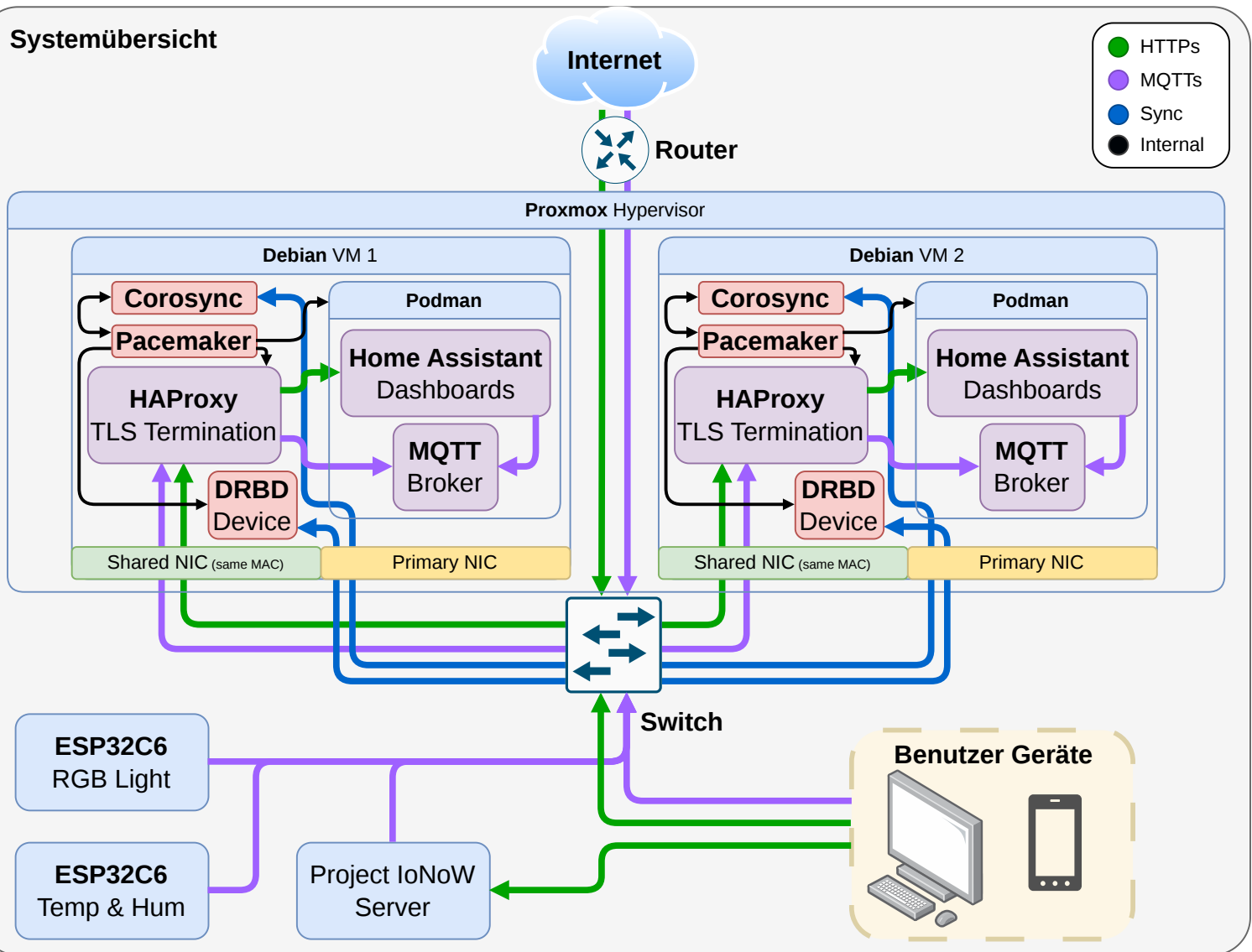


Systemübersicht



Projektarbeit - Hochverfügbare IoT Systeme: Phase 1

Das Ziel der ersten Projektarbeitsphase ist es eine redundante **IoT Cloud** Lösung zu entwickeln. Hierbei sollen zwei Server einen aktiv/passiv Redundanzcluster bilden, um selbst bei einem begrenzten Ausfallszenario, weiterhin den vollen Service anbieten zu können.

Für die Kommunikation wurde **MQTT** mittels dem **Mosquitto Broker** gewählt, da es die einfachste und populärste Methode ist. Für die Redundanz auch wichtig, da Mosquitto mittels der autosave Funktion in-memory Daten auch am Filesystem abspeichern kann und dadurch perfekt geeignet ist für einen aktiv/passiv Betrieb. EMQX wurde ebenfalls berücksichtigt, es ist aber ausschließlich aktiv/aktiv betriebsfähig. Mosquitto's MQTT-Bridge Funktionalität wurde ebenfalls betrachtet, diese ist aber nicht einsetzbar, wenn die Datenbank bereits mittels DRBD synchronisiert wird, da die passive Instanz niemals gleichzeitig laufen darf.

Für die historische Datenverwaltung sowie Datenvisualisierung wurde **Home Assistant** gewählt, da es die häufigste Lokal-Lösung ist und eine Vielfalt an Funktionalität mit geringstem Einrichtungsaufwand anbietet. Home Assistant soll außerdem auch die Verantwortung für das ausführen von Automationen übernehmen.

Als klassischer Sensor & Aktuator werden **ESP32C6** Mikrocontroller eingesetzt. Ein Temperatur und Luftfeuchtigkeitssensor und ein RGB-fähiges LED-Licht. Diese Werte werden historisch aufgezeichnet, bieten schöne Dashboards und können zum Präsentieren der Automationsfähigkeit fantastisch genutzt werden.

Für das Darstellen der Redundanzfähigkeit wird **Project IoNow** eingesetzt. Es ist eine zeitkritische MQTT-Anwendung, entwickelt in dem letzten Semester, wo auch nur 50 Millisekunden an Verzögerung, wie etwa bei einem Failover, klar zu spüren sind. Mehr Details dazu können auf [GitHub](#) gefunden werden.

Der Zugriff auf die Server wird über **HAProxy**, agierend als eine **Reverse-Proxy**, implementiert. MQTT sowie Home Assistant werden unter eigenen Benutzern als rootless **Podman** Container betrieben. Die **SSL/TLS Zertifikate** für die sichere Kommunikation werden kostenfrei von [Lets-Encrypt](#) erworben.

Für die Verwaltung der Cluster-Ressourcen und die Betätigung eines Failovers wird die klassische **pacemaker-corosync** Kombo genutzt. Dabei wird das **DRBD** Blockdevice mit dem Filesystem aktiviert, die Podman Container werden hochgefahren, und das Shared NIC wird aktiviert. Für die einfachere Verwaltung der Maschinen und ein leichtes Nachbauen werden **Proxmox** VMs statt physischen Geräten genutzt. Um das **Quorum-Problem** eines two-node Clusters etwas zu umgehen, wird das **STONITH**-Verfahren eingesetzt, um **Split-Brain** Szenarien zu minimieren.